

Totem 用户手册

Totem 用户手册

1. 从头开始

- 1.1 安装
- 1.2 启动数据库
- 1.3 创建数据库
- 1.4 访问数据库
- 1.5 数据库配置

2. 数据库使用

- 2.1 介绍
- 2.2 代理的基本概念
- 2.3 创建基本类
- 2.4 创建代理类
 - 2.4.1 定义select代理类
 - 2.4.2 定义join代理类
 - 2.4.3 定义union代理类
 - 2.4.4 定义group代理类
 - 2.4.5 定义宽松代理类
- 2.5 向类中添加对象
 - 2.5.1 向基本类追加对象
 - 2.5.2 向宽松代理类追加对象
- 2.6 查询
 - 2.6.1 普通查询
 - 2.5.2 跨类查询
- 2.7 更新对象
- 2.8 删除对象

3. 数据库维护

- 3.1 日常清理
- 3.2 备份与恢复

欢迎阅读TOTEM用户精简版手册。该手册是为了用户快速使用totem数据库撰写的文档。以下章节向用户介绍TOTEM对象代理数据库安装、使用等等。

1. 从头开始

推荐在下列配置中使用totem数据库：

操作系统：Ubuntu 18.04 LTS 64位
内存：8G RAM
硬盘：20G

1.1 安装

在开始使用Totem之前，首先需要进行安装。Totem可以由任何非特权用户安装，并不需要超级用户(root)的权限。以ubuntu系统为例安装totem。具体步骤如下：

- (1) 创建totem用户

运行时应该专门为该数据库设置一个专用用户名，例如totem，作为TOTEM的超级用户。输入命令：

```
adduser totem
```

以root、bin或者其它特殊权限的用户身份运行时存在安全问题。实际上tmaster拒绝以root身份运行。

(2) 下载deb包

从[珞珈图腾实验室网站](#)下载totem的deb安装包。

(3) 安装

```
sudo dpkg -i totem_1.0-1_amd64.deb
```

默认的安装位置为：/usr/local/totem

(4) 修改目录权限

将/usr/local/totem权限进行修改

```
cd /usr/local
sudo chown -R totem totem
```

完成以上步骤之后，totem数据库系统已经安装完成了。还需要初始化数据库，才能够启动数据库服务。

1.2 启动数据库

在启动数据库之前需要初始化数据库目录。在/usr/local/totem目录下创建Totem的data目录，并进行初始化：

```
mkdir /usr/local/totem/data
/usr/local/totem/bin/initdb -D ../data #在totem用户下执行
```

数据库目录初始化完成之后，使用totemctl命令启动数据库，用tsql来测试totem启动是否成功。

```
./totemctl -D /home/lnz/totem_data -l logfile start #启动数据库服务
./tsql -d postgres #连接数据库
```

使用tsql连接成功后，如上图所示。

1.3 创建数据库

看看你能否访问数据库服务器的第一个例子就是试着创建一个数据库。一台运行着的TOTEM服务器可以管理许多数据库。通常我们会为每个项目和每个用户单独使用一个数据库。你的节点管理员可能已经为你创建了可以使用的数据库。他应该已经告诉你这个数据库的名字。如果这样你就可以省略这一步，并且跳到下一节。要创建一个新的数据库，在我们这个例子里叫mydb，你可以使用下面的命令：

```
$ ./createdb mydb #在/usr/local/totem/bin下执行该命令
```

它应该生成下面这样的响应：

```
lnz@lnz-VirtualBox:/usr/local/totem/bin$ ./createdb mydb
CREATE DATABASE
```

如果这样，那么这一步就成功了，你就可以忽略本节余下的部分了。

如果看到类似的消息是：

```
createdb: could not connect to database totem: could not connect to server:
No such file or directory
Is the server running locally and accepting
connections on Unix domain socket "/tmp/.s.PGSQL.5432"?
```

这意味着该服务器没有启动，或者没有在createdb预期的地方启动。同样，你也要检查安装指导或者找管理员。

另外一个响应可能是这样：

```
createdb: could not connect to database totem: FATAL: user "joe" does not exist
```

在这里提到了你自己的登陆名。如果管理员没有为你创建TOTEM用户帐号，就会发生这些现象（TOTEM 用户帐号和操作系统用户帐号是不同的）。你需要变成安装TOTEM的操作系统用户的身份（通常是totem）才能创建第一个用户帐号。也有可能是赋予你的TOTEM用户名和你的操作系统用户名不同； 这种情况下，你需要使用 -U 开关或者使用PGUSER环境变量声明你的TOTEM用户名。

如果你有个数据库用户帐号，但是没有创建数据库所需要的权限，那么你会看到下面的东西：

```
createdb: database creation failed: ERROR: permission denied to create database
```

并非所有用户都经过了创建新数据库的授权。如果 TOTEM 拒绝为你创建数据库，那么你需要让节点管理员赋予你创建数据库的权限。出现这种情况时请咨询你的节点管理员。如果你自己安装了TOTEM，那么你应该以你启动数据库服务器的用户身份登陆然后参考手册完成权限的赋予工作。

你还可以用其它名字创建数据库。TOTEM允许你在一个节点上创建任意数量的数据库。数据库名必须是以字母开头，最大长度31个字符。一个方便的做法是创建和你当前用户名同名的数据库。许多工具假设该数据库名为缺省数据库名，所以这样可以节省你的工作量。要创建这样的数据库，只需要键入

```
$ ./createdb
```

如果你再也不想使用你的数据库了，那么你可以删除它。比如，如果你是数据库mydb的所有人（创建人），那么你就可以用下面的命令删除它：

```
$ ./dropdb mydb
```

（对于这条命令而言，数据库名不是缺省的用户名。这样你就必须声明它。）这个动作物理上将所有与该数据库相关的文件都删除并且不可取消，因此做这件事之前一定要想清楚。

1.4访问数据库

一旦你创建了一个数据库，你可以通过下面方法访问它：

运行TOTEM终端监控程序tsql，你可以在其中输入、编辑和执行SQL命令。接下来以之前的mydb为例，介绍如何运行 tsql。输入命令 ./tsql mydb，将看到如下响应。

```
Welcome to tsql 2.0, the Totem interactive terminal.

Type:  \copyright for distribution terms
       \h for help with SQL commands
       \? for help with tsql commands
       \g or terminate with semicolon to execute query
       \q to quit

mydb=# █
```

mydb=# 这个提示符表明你已经进入终端监控器并可以键入SQL查询到由终端监控器管理的工作区中。监控程序tsql会响应由反斜杠字符'\`开头的特殊命令。

比如，你可以键入下面命令获得各种TOTEM SQL命令的语法帮助：

mydb=# \h

```
mydb=# \h
Available help:
ABORT                CREATE LANGUAGE      DROP VIEW
ALTER AGGREGATE       CREATE OPERATOR CLASS END
ALTER CONVERSION      CREATE OPERATOR      EXECUTE
ALTER DATABASE        CREATE ROLE           EXPLAIN
ALTER DOMAIN          CREATE RULE           FETCH
ALTER FUNCTION        CREATE SCHEMA         GRANT
ALTER GROUP           CREATE SEQUENCE       INSERT
ALTER INDEX           CREATE TABLE         LISTEN
ALTER LANGUAGE        CREATE TABLE AS     LOAD
ALTER OPERATOR CLASS  CREATE TABLESPACE   LOCK
ALTER OPERATOR        CREATE TRIGGER        MOVE
ALTER ROLE            CREATE TYPE           NOTIFY
ALTER SCHEMA          CREATE USER           PREPARE
ALTER SEQUENCE        CREATE VIEW           PREPARE TRANSACTION
ALTER TABLE          DEALLOCATE            REINDEX
ALTER TABLESPACE     DECLARE              RELEASE SAVEPOINT
```

一旦你完成向工作区的查询输入，你可以键入下面内容将你的工作区提交给服务器：

mydb=> \g

这句话告诉服务器处理查询。如果你的查询以分号结尾，就不必使用"\g"命令；tdsql 将自动处理由分号结尾的查询。

要想从文件里读入查询，比如说myFile，而不是在命令行上输入，则键入：

```
\i myFile
```

从tsql中退出回到UNIX，键入

```
\q
```

则tsql将退出并回到你的命令行上。（有关更多脱字符的信息，在提示符上键入\?）。空白（比如，空格，tabs 和新行等）可以自由地在SQL查询里使用。单行注释使用"--"。该行所有在这个符号后面的内容都将被忽略。多行注释或行内注释，使用"/* ... */"。

1.5 数据库配置

配置文件名称为totem.conf，位于数据库集群目录下。它包括监听的ip和端口、数据库log设置、内存参数设置（共享内存，临时内存）。根据应用的需要在该配置文件下进行修改。

客户端认证是由一个配置文件(通常名为totem_hba.conf)控制的，它存放在数据库集群的数据目录里。HBA的意思是"host-based authentication"，也就是基于主机的认证。在initdb初始化数据目录的时候，它会安装一个缺省的totem_hba.conf文件。

totem_hba.conf文件的常用格式是一组记录，每行一条。空白行将被忽略，井号#开头的注释也被忽略。记录不能跨行存在。一条记录是由若干用空格和/或制表符分隔的字段组成。如果字段用引号包围，那么它可以包含空白。在数据库、用户或地址文件中引用一个关键词（如，all 或 replication）使这个词失去它的特殊含义，只是用这个名字匹配一个数据库、用户或主机。

每条记录声明一种连接类型、一个客户端 IP 地址范围(如果和连接类型相关的话)、一个数据库名、一个用户名字、对匹配这些参数的连接使用的认证方法。第一条匹配连接类型、客户端地址、连接请求的数据库名和用户名的记录将用于执行认证。这个处理过程没有 "跨越"或者"回头"的说法：如果选择了一条记录而且认证失败，那么将不再考虑后面的记录。如果没有匹配的记录，那么访问将被拒绝。

每条记录可以是下面七种格式之一：

```
local      database user auth-method [auth-options]
host       database user address auth-method [auth-options]
hostssl    database user address auth-method [auth-options]
hostnossl  database user address auth-method [auth-options]
host       database user IP-address IP-mask auth-method [auth-options]
hostssl    database user IP-address IP-mask auth-method [auth-options]
hostnossl  database user IP-address IP-mask auth-method [auth-options]
```

2.数据库使用

2.1介绍

本章提供一个如何在TOTEM中使用SQL执行简单操作的概述，并非完整的对象代理SQL语言的教程。在随后的例子里，我们假设你已经创建了名为myddb的数据库，并且已经启动了 tsq1。

TOTEM是一种基于对象代理模型的数据库管理系统。把数据存储在建表的概念已经快成了固有的常识了，这是关系数据库最基本的概念之一；但是还有其它的一些方法用于组织数据库。在类 Unix 操作系统上的文件和目录就形成了一种层次数据库的例子。更现代的发展是面向对象的数据库。与传统面向对象模型相比，对象代理模型更具柔软性，操作更灵活，使用也更方便。第3章将对“代理”进行更细致的描述。

2.2 代理的基本概念

所谓“代理”，是指对一个对象（源对象）的所有或部分属性进行一定的切换（switching）操作，如换名、运算等，而衍生出另外一个对象（代理对象）的过程。通过对象的代理，用户可以将不同的对象进行分割组合，形成新的视图形式的对象。与原来的面向对象模型相比，对象代理模型更具柔软性，操作更灵活，使用也更方便。

源对象和代理对象都属于各自的类。因而形成了源类和代理类的概念。拥有切换操作和模式的代理对象被聚集在一个代理类中。代理类对源类的“代理”和一般意义上的“继承”是有区别的。代理类更像是源类的视图，不同的代理对象表现出了源对象扮演的不同角色。一个源类上可以同时定义多个代理类。代理类也不仅限于代理一个源类，它可以通过连接或并操作同时在几个源类上进行代理。代理类上也可以再定

义其它的代理类。代理关系是一个有向图的形状。位于代理层次最顶端的类称为基本类，它们可以看作是没有源类的代理类。（本文中，基本类和代理类统称为“类”）

在存储模式上，基本类的对象占有存储空间，存储实际数据。数据库中大部分数据都存储在基本类中。代理对象中的代理属性（称为“虚属性”）只拥有模式而没有实际的物理数据元。源对象和代理对象之间由双向指针来联系，并且由切换操作来确定代理对象中虚属性的值。代理对象可以扩展属于自己属性（称为“实属性”），这些扩展属性才真正占有存储空间。

代理类的定义有点类似于关系数据库中视图的定义。在一般的查询操作上，代理类和普通的类完全一样。

###

2.3 创建基本类

TOTEM中创建一个基本类或代理类，对应传统关系型数据中创建一个表。基本类是没有源类的类，它们是最基本的类，其所有的属性都是实属性。其本质上和关系表一样，但是它拥有方法。因此我们定义一个基本类的命令和关系数据库中定义基本表的命令比较相似，包括：定义类名，属性名和属性类型；建立约束，如属性是否允许为空，属性的缺省值等；创建方法，定义方法的名称，参数类型，返回类型以及函数体。

例如定义一个基本类faculty的SQL语句如下，该类存储所有教师的信息。

```
CREATE CLASS faculty
(
  Id INT NOT NULL UNIQUE,
  Name VARCHAR(20) NOT NULL,
  birth INT,
  department VARCHAR(30),
  salary INT
);
```

2.4 创建代理类

定义一个代理类需要一个代理规则（本质上是一个查询语句）来定义对源类进行的形式上的变换。为了保证代理关系的明晰，必须对代理的规则进行一定的限定。在对象代理模型中，只允许四种简单的代理规则。分别为select, join, union和group代理。本节以SELECT代理类为例进行描述，更多细节详见第5章。

定义代理规则时可以加上一定的语义约束，使得被代理的对象符合某种条件。当对象被更新时，系统会根据这些条件自动维护对象之间的一致性。如果不声明语义约束，则定义了一个宽松代理类（见本节后续内容），它其中的对象是由用户手动维护的。

2.4.1 定义select代理类

Select型代理只允许从一个类中选择源对象进行代理。源对象和代理对象是一一对应的。

通过如下语句可创建一个2.2小节中faculty类的选择代理类。

```
CREATE SELECTDEPUTYCLASS computer_faculty (field VARCHAR(30))
AS
(SELECT id, name, (extract(year from current_date) - birth) AS age FROM faculty
WHERE department = 'computer')
WRITE (faculty.birth = 'extract(year from current_date)-age');
```

computer_faculty代理类直接继承了源类faculty中的id,name属性, 而且增加了一个属性field用来表示教师的专业领域。并且通过表达式(cextract(year from current_date)-birth) AS age 将birth属性的值转换为age属性继承下来。约束条件WHERE department ='computer'声明了该代理类只代理faculty类中属于computer部门的对象。WRITE子句定义了age虚属性的写切换操作。通过它可以更新age属性。

定义后如果进行如下查询:

```
mydb=# select * from computer_faculty;
 id | name | age | field
----+-----+----+-----
  2 | john | 32  |
  1 | jack | 31  |
(2 rows)
```

可以看到, 只有computer部门的教师被选择出来了。age属性的值与birth的值对应。field域暂时为空, 因为该域没有代理任何源类数据, 只能在代理类创建后用更新命令输入其值。如下:

```
UPDATE computer_faculty SET field = 'Database' WHERE name = 'jack';

UPDATE computer_faculty SET field = 'Network' WHERE name = 'john';
```

再次查询该类则返回如下结果

```
mydb=# select * from computer_faculty;
 id | name | age | field
----+-----+----+-----
  2 | john | 32  | Network
  1 | jack | 31  | Database
(2 rows)
```

技巧: 代理类中的“虚属性”实际上并没有被物化, 所以不会占用代理类的存储空间。所以尽量将要重用的数据放在源类中, 再通过不同的代理类来使用它们。利用转换表达式可以将源属性的表现形式进行改变。

2.4.2 定义join代理类

Join型代理是在几个类的连接结果上进行代理。它可以达到类似于“多继承”的表现效果。一个代理对象在每个源类中都对应一个源对象。

例子: 创建一个代理类assistant存储所有学生(既是学生又是老师)的信息。

```
CREATE JOINDEPUTYCLASS assistant
AS
( SELECT student.name, salary FROM student,faculty
  WHERE student.id = faculty.id );
```

查询结果:

```
mydb=# select * from assistant;
 name | salary 
-----+-----
 john |  20000 
(1 row)
```

可以看到assistant对象同时具有了两个源类的属性。

2.4.3 定义union代理类

union型代理相当于把若干个select代理的结果并在一个代理类中。

例子：创建student和faculty的union型代理类person，将两类对象抽象为一种同模式的对象。

```
CREATE UNIONDEPUTYCLASS person
AS
( SELECT name, birth FROM student UNION
  SELECT name, birth FROM faculty);

select * from person;
```

查询结果：

```
postgres=# select * from person;
 name | birth 
-----+-----
 mike |  1980 
 bill |  1982 
(2 rows)
```

可见person类的对象集合是student和faculty类的并集。

2.4.4 定义group代理类

Group型代理是在一个类上进行分组的代理。虽然和select代理一样，它要求一个代理类只对应一个源类，但是一个代理对象代理一组源对象。Group型代理规则中一般含有聚集操作。

例子：创建faculty的group代理类

```
CREATE GROUPDEPUTYCLASS dept_size
AS
( SELECT COUNT(id) AS size, department AS dept_name FROM faculty group by
  department);
```

这个代理类统计各个学校部门的教师总数。执行查询

```
select * from dept_size;
```

返回：

```
postgres=# select * from dept_size;
 size | dept_name
-----+-----
    1 | math
    2 | computer
(2 rows)
```

2.4.5 定义宽松代理类

前几节中定义的代理类都是含有语义约束条件的，称为严格的代理类。严格代理类一旦定义，其中的对象集就由系统根据约束条件自动确定了，不能由使用者任意的插入或删除。代理规则中不含有语义约束的代理类称为宽松代理类。定义后，宽松代理类中的对象集由用户来手动维护。

宽松代理类的代理规则和严格代理类的区别是：1) 必须写IMPRECISE关键字。2) 不能声明任何语义约束。（注意：在严格代理类的定义中，也可以不声明语义约束，但是这种定义隐含表示了所有对象都必须被代理。而宽松代理类中禁止声明任何语义约束。）

宽松代理类的代理规则和严格代理类的区别是：1) 必须写IMPRECISE关键字；2) 不能声明任何语义约束。在严格代理类的定义中，也可以不声明语义约束，但是这种定义隐含表示了所有对象都必须被代理。而宽松代理类中禁止声明任何语义约束。

以下给出创建student类的非严格select代理类的例子。该类用来存储“我的朋友”的信息。显然student类中哪些对象可以成为“我的朋友”是由用户来确定。

```
CREATE IMPRECISE SELECTDEPUTYCLASS my_friends(telephone int)
AS ( SELECT name FROM student);
```

2.5 向类中添加对象

TOTEM向类中追加对象分为两种情况：（1）在基本类中创建新对象；（2）向宽松代理类追加对象。

2.5.1 向基本类追加对象

用INSERT命令在基本类中创建新对象，语法结构如下：

```
INSERT INTO class [ ( column [, ...] ) ]
    { DEFAULT VALUES | VALUES ( expression [, ...] ) | SELECT query }
```

INSERT语句输入为：

- (1) class, 现存的基本类的名称；
- (2) column, class中的字段名；
- (3) DEFAULT VALUES, 所有字段都会用NULL或者创建时用DEFAULT子句声明的值填充；
- (4) expression, 赋予column的一个有效表达式或值；
- (5) query, 一个有效的查询。

INSERT语句输出为：

- (1) INSERT oid 1, 如果只插入了一个对象，返回此信息，其中oid是被插入对象的数字标识；
- (2) INSERT 0 # 如果插入了多个对象，返回此信息，其中#是插入对象的数目。

INSERT 允许我们向类中插入新对象。我们可以一次插入多个对象。目标列表中的列/字段可以按任何顺序排列。在目标列中没有出现的列/字段将插入缺省值：定义了DEFAULT值或者NULL。如果向定义为NOT NULL的列中插入NULL值，TOTEM将拒绝新对象。如果表达式不是正确的数据类型，将试图进行自动的类型转换。要想向类中插入数据，你必须有插入权限，同样也要有选择权限用于处理WHERE子句中声明的任何类。请注意，这里提到的“列/字段”对应对象的属性。

第一个INSERT的用法举例：向films 里插入一个对象。命令如下：

```
INSERT INTO films VALUES  
('UA502', 'Bananas', 105, '1971-07-13', 'Comedy', INTERVAL '82 minute');
```

第二个用法举例同样为向films里插入一个对象，里面省略了字段len，因此将存储len的缺省值NULL。命令如下：

```
INSERT INTO films (code, title, did, date_prod, kind)  
VALUES ('T_601', 'Yojimbo', 106, DATE '1961-06-16', 'Drama');
```

用户也可以将另一个类tmp中的对象插入films中，命令如下：

```
INSERT INTO films SELECT * FROM tmp;
```

2.5.2 向宽松代理类追加对象

用ADD INTO向宽松代理类中追加对象，该语句语法结构如下：

```
ADD ANY|INEXISTENT INTO name FROM source WHERE condition
```

ADD INTO命令只能用于宽松代理类。宽松代理类创建时不含有任何对象，如果用户希望将源类中的某些对象加入到宽松代理类时，则需要使用该命令。

ADD INTO语法解释如下：

- (1) ANY，不管where子句中选择的对象是否已经被代理过，都将它加入到代理类中，这样可以使得一个对象在代理类中对应多个代理对象；（
- (2) INEXISTENT，与ANY相反，如果发现where子句中选择的源对象在该代理类中已经存在了至少一个代理对象，则在这次ADD INTO命令中不再生成它的代理对象；
- (3) name，插入的目标类的名字；
- (4) source，源对象所属的源类名；
- (5) condition，用以选择源对象的布尔条件表达式。不可以含有子查询。

ADD INTO语句的输出为：

- (1) count，成功，返回加入对象的数目；
- (2) ERROR: can not insert into a precise deputy class，即TOTEM禁止使用ADD INTO向严格代理类加入对象。

2.6 查询

对象代理数据库的查询语言提供对类、代理类的查询支持，此外还提供基于路径表达式的跨类查询功能。对于普通的查询（指不涉及到代理类、代理对象的查询），对象代理数据库的查询处理过程可以沿用传统数据库的常用实现方法。但对于涉及代理类虚属性的查询处理，需要进行特殊的处理，使代理类的查询能采用与传统数据库的查询一致的形式描述和实现。对象代理数据库系统TOTEM建立了一套完善的查询机制，使得对于用户而言，可以自由、透明地查询类和代理类，另外还实现了特殊的导航式查询机制，即跨类查询。对象代理数据库中，（代理）对象之间的代理关系由物理地址指针链接来表示实现。因此，对象之间构成了由代理关系相连的对象层次结构。利用对象层次结构，可以从某个（代理）类的（代理）对象出发，查询与其直接或间接代理关联的其他（代理）类的（代理）对象。跨类查询利用了（代理）对象间形成的一个对象指针路径，并使用路径表达式来反映该路径的语法结构。

2.6.1 普通查询

一个查询是从数据库中检索数据的过程或命令，TOTEM使用SELECT命令声明查询。SELECT命令的通用语法是

```
SELECT select_list FROM class_expression [sort_specification]
```

接下来详细描述SELECT语法的各个部分。

最简单的查询的形式如下

```
SELECT * FROM class1;
```

假设有一个类叫做 class1，这条命令将从 class1 中检索所有对象的所有属性。选择列声明 * 意味着所有类表达式提供的属性。也可以选择可选列的一个子集甚至在检索它们之前对属性进行计算。假设class1有属性a、b和c，那么你可以用下面的查询：

```
SELECT a, b + c FROM class1;    #（假设b和c都是数字数据类型）
```

FROM class1是一种非常简单的类表达式（class_expression）。通常，类表达式可以是基本类、代理类、连接和子查询的复杂构造。但你也可以省略类表达式记录而用SELECT命令当做一个计算器，如SELECT 3 * 4。选择列表（select_list，上例中为a,b+c）里的表达式也可以返回变化的结果。比如可以用这个方法调用函数。

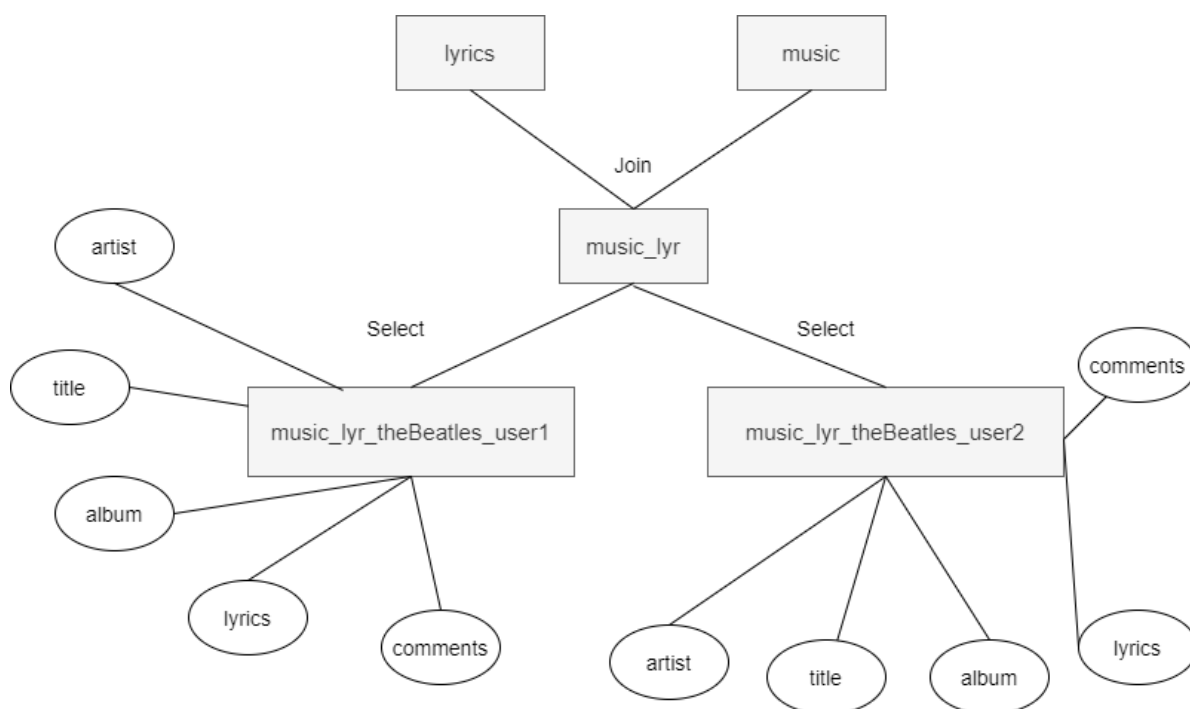
```
SELECT random();
```

类表达式（class_expression）声明一个类。SELECT a, b + c FROM class1例子中的类表达式非常简单，仅包含一个类class1。

实际上类表达式可以根据需要选用WHERE，GROUP BY，和HAVING等子句。这些子句声明一系列对源自FROM子句的类（即要查询的类）的转换操作，转换生成的类用于产生输出，而输出根据选择列表select_list生成。类表达式支持对类的复杂转换，比如连接类。一个连接类是根据特定的连接类型的规则从两个其它类中派生的类，具体而言TOTEM支持INNER、OUTER和CROSS JOIN连接类型。

2.5.2 跨类查询

为了清晰描述跨类查询，这里给出一个详细的例子。同样考虑再跨媒体数据管理场景下，有用户希望看到其他用户对某首歌曲的评价，那么首先给出该场景下的模式图，如下图所示：



music_lyr_TheBeatles_user1、music_lyr_TheBeatles_user2都是Music_lyr的代理类；Music_lyr存储歌词和歌曲，其代理类存储用户评价、专辑和歌词等信息。如果用户user1想看到其他用户user2对相同的属于专辑“TheBeatles”的歌曲的评价，通过简单的跨类查询即可实现，具体的SQL语句如下：

```
SELECT (music_lyr_TheBeatles_user1-> music_lyr_TheBeatles_user2).comments
FROM music_lyr_TheBeatles_user1
WHERE title='TheOne';
```

注意：在使用跨类查询的时候需要使用括号将所含的路径包括进去，同时无法使用*来查询，例如：

```
#未使用括号
SELECT music_lyr_TheBeatles_user1-> music_lyr_TheBeatles_user2.comments
FROM music_lyr_TheBeatles_user1
WHERE title='TheOne';
#无法使用*查询所有属性
SELECT (music_lyr_TheBeatles_user1-> music_lyr_TheBeatles_user2).*
FROM music_lyr_TheBeatles_user1
WHERE title='TheOne'
```

2.7 更新对象

用户可以用UPDATE命令更新现有的对象。假设你发现所有11月28日的温度计数都低了两度，那么你就可以用下面的方式更新数据：

```
UPDATE weather
SET temp_hi = temp_hi - 2, temp_lo = temp_lo - 2
WHERE date > '1994-11-28';
```

对象的新状态如下所示。

```
SELECT * FROM weather;
city    | temp_lo | temp_hi | prcp | date
-----+-----+-----+-----+-----
San Francisco | 46 | 50 | 0.25 | 1994-11-27
San Francisco | 41 | 55 | 0 | 1994-11-29
Hayward | 35 | 52 | | 1994-11-29
(3 rows)
```

2.8 删除对象

用户可以用DELETE命令从类中删除对象。假设你对Hayward的天气不再感兴趣，那么你可以用下面的方法把那些对象从类中删除：

```
DELETE FROM weather WHERE city = 'Hayward';
```

所有属于Hayward的天气记录都将被删除。

```
SELECT * FROM weather;
city    | temp_lo | temp_hi | prcp | date
-----+-----+-----+-----+-----
San Francisco | 46 | 50 | 0.25 | 1994-11-27
San Francisco | 41 | 55 | 0 | 1994-11-29
(2 rows)
```

我们用下面形式的语句的时候一定要小心

```
DELETE FROM classname;
```

如果没有条件，DELETE将从指定类中删除所有对象，把它清空。做这些之前系统不会请求你确认！

3.数据库维护

3.1 日常清理

由于数据库长期运行，会引起大量的垃圾数据和旧数据的丢失。为了保持所建数据库的平稳运行，我们必须做一些维护工作。这里所讨论的工作主要是恢复磁盘空间、更新规划器统计、维护日志文件以及回收OID。这些工作都是通过vacuum命令实现。所以，系统管理员需要定期的运行vacuum，使得数据库能够运行的更愉快。

在正常的数据库操作里，对一行的UPDATE或DELETE并未立即删除旧的元组(行的版本)。这个方法对于获取多版本并行控制的好处是必要的。但是经过长时间后，任何事务都不会关心这些已经过期的数据，所以回收这些记录所占物理空间是很必要的。

清理时，你可以定义清理某个类（这主要是针对那些经常进行删除更新）这种情况的代价比较小，可以设置一个周期性的 cron 任务清理那些选定的类，而忽略那些已经知道变化比较少的类；第二种方式是对整个数据库的所有类进行清理。在这种情况下，清理工作进行的比较完全但是代价比较大，需要将整个数据库冻结起来，所以我们推荐在一天中的低使用的时段安排一次整个数据库的 VACUUM。如果系统中有多个数据库，别忘了分别对每个数据库进行VACUUM（参考vacuumdb 脚本）。

标准的VACUUM，维护磁盘的稳定状态：找出旧数据，并将它所占据的物理空间标志为可用。

VACUUM FULL，除了执行标准VACUUM的工作外，还进行整个类的磁盘清理，并归还所有的空闲空间给操作系统。一般来说，对于那些更新频繁的类，中等频度的VACUUM 比低频度的VACUUM FULL更合适。对于一个类中大部分的删除元组，我们建议使用 VACUUM FULL，这样该类的稳定态尺寸可以因为VACUUM FULL更富侵略性的方法而显著减小。

3.2 备份与恢复

SQL 转储的方法采用的主意是创建一个文本文件，这个文本里面都是 SQL 命令，当把这个文件回馈给服务器时，将重建与转储时状态一样的数据库。TOTEM 为这个用途提供了应用工具 totem_dump。这条命令的基本用法是：

```
totem_dump dbname > outfile
```

正如你所见，totem_dump 把结果输出到标准输出。我们下面就可以看到这样做有什么好处。totem_dump 是一个普通的 TOTEM 客户端应用，你可以通过这个命令将一个数据库中的数据备份出来，但是你必须拥有这个数据库所有内容的读权限，一般是要求数据库管理员来作这些事情。使用 totem_dump --help 查看更多totem_dump用法。